

Ram Mohan Rao

Silicon has been singularly successful as a computing material. So much so that there's even an area nicknamed after the element—which, as some have half-jokingly pointed out, should be called "iron oxide valley" instead of "silicon valley" because of the disk industry. Every desktop computer in the world has silicon in it; and we can't imagine computing except in the context of silicon.

Is there something fundamental about silicon that makes it impossible to live without in the computing world? Nothing, really. Silicon and its properties just happen to be extremely suitable for computers to be built upon. But it's not that other materials—such as DNA, for example—cannot be explored.

DNA? When one first hears about "DNA Computing," the natural reaction is bewilderment. It needn't be: one just needs to know what "computing" really means. "Computing" does not mean browsing the Web or playing an MP3 song. Computing means the manipulation of numbers or other symbols, while having a means of feeding inputs to, and extracting outputs from, the system that is doing the manipulation.

We decided a long time ago that the best things to manipulate were ones and zeroes (binary), and that the ideal devices to do that were to be silicon-based. That's about the hardware. What about the software—the stuff that does the manipulation? John von Neumann—the pioneer of the stored-program concept—decided that the software, too, should be put into the silicon, and there you have it.

Now substitute A, C, G and T for the ones and zeroes, substitute enzymes for the software, and you have a DNA computing system. To expand this a little bit: A, C, G and T are the nucleotides (or bases) in DNA molecules, one of the fundamental building blocks of life. The enzymes we're talking about are chemicals that do things to DNA in much the same way they do in your body. And since software needs to 'operate on' hardware, we need to have a basic set of operations. That, too, is provided by DNA: the molecules can split, copy themselves, recombine, and so on, which gives us an adequate set of operations—just as adding, XORing, and so on give us an adequate set of operations for ones and zeroes.

Note that we haven't mentioned a "DNA computer" so far. There's a good reason for that: DNA computers don't exist. DNA computing does. How? Well, we think of computers as things that produce results 'on their own,' given a set of inputs. Like when you type in "thinkdigit.com" into a browser, you don't need much further inspection of the system before your browser takes you there. On the other hand, when we talk about "DNA computing," we're talking about solving problems using DNA—with or without human intervention—and this was Leonard Adleman's

Is there something fundamental about silicon that makes it impossible to live without in the computing world? Nothing, really

idea. Adleman is a theoretical computer scientist and professor of computer science and molecular biology at the University of Southern California.

The Travelling Salesman Problem

Adleman solved a broad version of the Travelling Salesman Problem (the TSP) using DNA. The TSP is quite simple to understand, really (but difficult to solve in most cases). Think of a salesman who needs to do a tour of some cities—say five, including the origin city. He needs to visit each city once, and return to his origin city. He shouldn't visit any city twice or more—just once.

The problem is to find the shortest route by which he can complete his tour, so he can save on the airfare. For example, say he starts from Mumbai; he needs to visit Delhi, Kolkata, Chennai and Bangalore, with Kolkata being his last leg, and return to Mumbai. What is the shortest route? You can see there are several possibilities. He could do M-D-B-C-K, or he could do M-C-D-B-K, or he could do...

Since we know some Indian geography, it's easy to see that our first option is the best. When the problem is limited to a few cities, it can be solved by hand; when it involves, say, 20 cities, a computer would take a while to do it. When there are, say, 200 cities, a supercomputer is needed, doing it the 'brute force' way—looking up each route. And when it involves 2,00,000 cities, the fastest supercomputer on earth probably can't do it in a hundred years!

So did Adleman do it? Did he use his DNA—rather, the DNA in his test tube—to solve what supercomputers can't? Actually, no. He solved something like our example above, which we can do by hand. It was groundbreaking because he illustrated what could be done using DNA. That's needed because, as you've probably noticed from the figures above, the complexity of the problem explodes as the number

of cities goes up. That's because of the massively parallel nature of the problem—meaning that the computer (or computing device) cannot work out a neat, systematic way of exploring routes. It has to look at many, many paths at the same time, one of which will turn out to be the answer. (There happen to be algorithms that simplify the process heavily, but that, again, depends on human ingenuity—change the problem and it'll take people years to discover a cooler way to solve it.)

That's where DNA computing comes in. The hardware—the DNA molecules themselves—is used to encode the operation, and in one droplet of DNA solution are more than a trillion molecules. They all work in parallel, doing the same kind of thing, and then some molecules morph into the answer.

The Key Insight

Like we said, Adleman kickstarted the field of DNA computing by using DNA to solve a broad version of the TSP. What he did was pour a solution with some DNA into a test tube, then add some enzymes to it, then add more enzymes, and